GitHub Trending Top 10

每日热门开源项目深度解读

2026-04-17

今日榜单

1. **EvoMap/evolver** — JavaScript
   The GEP-Powered Self-Evolution Engine for AI Agents. Genome Evolution Protocol. ... — NEW

2. **lsdefine/GenericAgent** — Python
   Self-evolving agent: grows skill tree from 3.3K-line seed, achieving full system... — 2天

3. **SimoneAvogadro/android-reverse-engineering-skill** — Shell
   Claude Code skill to support Android app's reverse engineering — NEW

4. **BasedHardware/omi** — Dart
   AI that sees your screen, listens to your conversations and tells you what to do — NEW

5. **Lordog/dive-into-llms** — Jupyter Notebook
   《动手学大模型Dive into LLMs》系列编程实践教程 — 3天

6. **Donchitos/Claude-Code-Game-Studios** — Shell
   Turn Claude Code into a full game dev studio — 49 AI agents, 72 workflow skills,... — NEW

7. **jamiepine/voicebox** — TypeScript
   The open-source voice synthesis studio — 4天

8. **lukilabs/craft-agents-oss** — TypeScript
   NEW

9. **Tracer-Cloud/opensre** — Python
   Build your own AI SRE agents. The open source toolkit for the AI era ✨ — NEW

10. **obra/superpowers** — Shell
    An agentic skills framework & software development methodology that works. — NEW

#1 JavaScript NEW

evolver

The GEP-Powered Self-Evolution Engine for AI Agents. Genome Evolution Protocol. | evomap.ai

Stars **3,511** Forks **363** Today **+812**

https://github.com/EvoMap/evolver

项目简介：Evolver 是一个基于 GEP（基因组进化协议）的 AI 智能体自我进化引擎。它旨在解决 AI 智能体开发中“提示词（Prompt）调整”过程零散、难以复用和审计的问题，通过一套协议化的方法，将临时的提示词调优转变为可审计、可复用的进化资产。

核心功能：1. **自动日志分析**：扫描运行时日志和记忆文件，识别错误模式和信号。2. **协议化进化**：依据 GEP 协议，从预定义的“基因”或“胶囊”资产库中选择最佳匹配项，生成结构化的进化提示。3. **审计追踪**：记录每一次进化事件，确保变更过程可追溯。4. **多种运行模式**：支持单次运行、审查模式和后台守护进程模式。

适用场景：该项目主要面向需要规模化管理和迭代 AI 智能体提示词的团队，以及对进化过程的可审计性、确定性有严格要求的开发环境。它不适合仅需一次性脚本或追求完全自由形式、无协议约束的创意性变更的场景。

技术亮点：其核心在于将生物进化中的概念（如基因、突变）与软件工程实践（如版本控制 Git 用于回滚和影响分析）相结合，构建了一套约束性的进化协议。项目强调“生成提示而非直接修改代码”的安全模型，并通过与宿主运行时（如 OpenClaw）的集成，将进化指令输出为可被解释执行的标准化文本。

上手建议：建议从项目官网 evomap.ai 的文档和 Wiki 开始，了解 GEP 协议的核心概念。对于使用者，可按照 README 的“快速开始”部分克隆项目并运行 `node index.js` 体验基础流程；对于贡献者，应仔细阅读其安全模型和协议设计，理解项目“约束性进化”的哲学。

#2 Python 连续 2 天

GenericAgent

Self-evolving agent: grows skill tree from 3.3K-line seed, achieving full system control with 6x less token consumption

Stars **3,199** Forks **352** Today **+872**

https://github.com/lsdefine/GenericAgent

项目简介：GenericAgent 是一个极简、自进化的自主智能体框架。它旨在解决传统智能体框架依赖预定义技能、消耗大量计算资源的问题，通过让智能体在完成任务时自动将执行路径“结晶”为可复用的技能，实现从少量种子代码开始的自我能力成长。

核心功能：1. **自我进化**：每次解决新任务后，自动将执行流程固化为技能，形成不断生长的个人技能树。2. **极简架构**：核心代码仅约3000行，代理主循环约100行，依赖简单。3. **系统级控制**：通过9个原子工具直接控制浏览器、终端、文件系统、键鼠输入等。4. **高兼容性与高效**：支持主流大模型，通过分层内存管理将上下文窗口控制在3万Token以内，显著降低成本。

适用场景：适合希望构建高度个性化、能自动化处理复杂电脑操作（如数据抓取、软件操作、日常办公流程）的个人开发者或技术爱好者。它也适用于需要智能体长期学习并积累专属技能库的研究或自动化场景。

技术亮点：其“技能结晶”机制是核心创新，将动态探索过程转化为静态可调用技能，实现了能力的指数级积累。此外，它通过注入真实浏览器来保持登录会话，以及采用分层记忆确保相关上下文精准召回，在提升成功率的同时大幅降低了Token消耗。

上手建议：建议从项目README的“Quick Start”部分开始，按照标准安装步骤克隆仓库并配置API密钥，通过`launch.pyw`启动基础Web界面。可以先尝试一些简单的自动化指令，观察其如何探索并形成技能，再逐步深入自定义工具或贡献代码。

#3 Shell NEW

android-reverse-engineering-skill

Claude Code skill to support Android app's reverse engineering

Stars **2,506** Forks **271** Today **+375**

https://github.com/SimoneAvogadro/android-reverse-engineering-skill

项目简介：这是一个专为 Claude Code 设计的技能插件，旨在自动化 Android 应用逆向工程的核心流程。它主要解决了开发者和安全研究人员在缺乏源代码时，手动反编译、分析应用结构并提取网络 API 接口（如端点、认证信息）效率低下的痛点。

核心功能：1) 支持对 APK/XAPK/JAR/AAR 等多种格式文件进行反编译，并可选用 jadx 或 Fernflower/Vineflower 引擎进行对比；2) 智能提取并记录应用使用的 HTTP API，包括 Retrofit 接口、OkHttp 调用和硬编码 URL；3) 追踪从界面（Activity/Fragment）到网络请求的完整调用链路；4) 提供应对 ProGuard/R8 混淆代码的分析策略。

适用场景：主要面向移动安全研究员、渗透测试人员和需要对第三方 Android 应用进行 API 分析或安全审计的开发者。典型场景包括：分析应用通信协议、进行安全漏洞挖掘、理解竞品应用的网络接口设计，或为已下架应用重建其 API 文档。

技术亮点：其设计亮点在于将多个独立的逆向工程工具（jadx, Fernflower）和复杂的分析模式（API 模式匹配、调用流追踪）封装成一套简洁、可交互的 Shell 脚本工作流，并深度集成到 Claude Code 的对话环境中，实现了从自然语言指令到专业分析结果的“一键式”转换。

上手建议：新用户建议直接从 Claude Code 的插件市场安装，并通过 `/decompile` 命令配合一个测试 APK 快速体验完整工作流。若想深入了解或贡献，可仔细阅读项目 `references/` 目录下的指南文档，并从研究 `scripts/` 中的独立 Shell 脚本开始，理解其如何编排底层工具。

#4 Dart NEW

omi

AI that sees your screen, listens to your conversations and tells you what to do

Stars **9,463** Forks **1,646** Today **+378**

https://github.com/BasedHardware/omi

项目简介：Omi 是一个全平台、开源的“第二大脑”AI助手，旨在解决信息过载与上下文遗忘问题。它通过实时捕获用户的屏幕内容和对话，进行转录、总结并生成待办事项，构建一个能记住用户所见所闻的长期记忆AI聊天体。

核心功能：1. **全设备感知**：捕获桌面、手机及可穿戴设备的屏幕与音频。2. **实时转录与摘要**：利用语音识别（STT）实时转录音频，并生成内容摘要和行动项。3. **情境化AI记忆**：基于所有历史记录提供具备上下文的AI聊天功能。4. **多模态集成**：支持文本、语音、视觉信息的统一处理与检索。

适用场景：适合需要高效处理多任务和信息流的专业人士，如开发者、项目经理、研究者或学生。典型场景包括记录线上会议、总结工作流、管理碎片化知识，或在需要回顾复杂项目历史时快速获取上下文。

技术亮点：其架构体现了**异构技术栈的深度融合**：前端使用Swift（macOS）、Flutter（移动端）和Dart，后端核心为Python FastAPI，并集成了Rust（高性能桌面后端）、C（嵌入式固件）和多种云服务（如Firestore、Redis）。这种设计平衡了性能、开发效率与跨平台能力。

上手建议：普通用户可直接从官网下载各平台客户端试用。对于开发者，建议从 `./run.sh --yolo` 快速体验云端连接的macOS版本开始；若想深入参与，可查阅桌面端（Swift/Rust）或移动端（Flutter）的README，从相对独立的模块入手熟悉代码结构。

#5 Jupyter Notebook 连续 3 天

dive-into-llms

《动手学大模型Dive into LLMs》系列编程实践教程

Stars **31,217** Forks **3,789** Today **+1,385**

https://github.com/Lordog/dive-into-llms

项目简介：《动手学大模型》是一个由上海交通大学课程讲义拓展而来的开源编程实践教程。它旨在通过“动手”实践的方式，帮助学习者（尤其是学生和研究者）快速入门大语言模型（LLM）领域，解决从理论到实践的鸿沟问题，内容覆盖微调、部署、安全、应用等多个前沿方向。

核心功能：1. **全流程实践指南**：提供从模型微调、部署到API调用的完整操作教程。2. **前沿主题覆盖**：不仅涵盖提示工程、知识编辑等基础，还深入越狱攻击、模型水印、隐写术等安全与高级主题。3. **国产化支持**：特别提供了基于华为昇腾平台的《大模型开发全流程》公益教程，适配国产化生态。

适用场景：非常适合高校学生用于课程设计、学术研究入门，以及希望系统学习LLM实践技能的开发者。无论是想快速搭建一个可用的模型Demo，还是想深入研究大模型的安全机制与高级应用，都能从中找到对应的实践模块。

技术亮点：1. **“课件+教程+代码”三位一体**：每个主题都配有理论课件、文字教程和可运行的Jupyter Notebook脚本，学习路径清晰。2. **内容前沿且实用**：教程选题紧扣研究热点（如数学推理、GUI Agent），代码示例能直接运行验证，降低了前沿技术的上手门槛。3. **社区驱动与公益性质**：项目积极欢迎PR，并由高校课程背书，保证了内容的持续更新和质量。

上手建议：建议新手从第一章“微调与部署”或第二章“提示学习与思维链”开始，这是构建对大模型直观认知的基础。对于有特定兴趣的研究者，可直接根据目录选择对应主题，结合课件理论阅读并运行配套的`.ipynb`脚本进行实践。有意贡献者可以关注项目的`Issues`或对现有教程进行补充优化。

#6 Shell NEW

Claude-Code-Game-Studios

Turn Claude Code into a full game dev studio — 49 AI agents, 72 workflow skills, and a complete coordination system mirroring real studio hierarchy.

Stars **11,395** Forks **1,688** Today **+1,107**

https://github.com/Donchitos/Claude-Code-Game-Studios

项目简介：这是一个为 Anthropic 的 Claude Code 设计的 AI 代理协作框架，旨在将单个 AI 对话会话转变为一个结构化的虚拟游戏开发工作室。它解决了使用通用 AI 助手进行复杂项目（如游戏开发）时缺乏流程、分工和质量控制的问题，通过模拟真实工作室的层级和职责来提升产出的组织性与质量。

核心功能： 1. **层级化代理系统**：提供 49 个具有明确职责的 AI 代理，分为总监、部门主管和专家三个层级，模拟从创意、技术到 QA 的完整开发生命周期。 2. **标准化工作流与技能**：内置 72 个“斜杠命令”技能，覆盖从项目启动、系统设计、任务分解到开发验收的全流程，引导用户按规范操作。 3. **自动化质量保障**：通过 12 个钩子和 11 条规则，在代码提交、资产变更等环节自动执行验证，并针对不同代码路径（如游戏逻辑、UI）强制执行编码标准。 4. **多引擎支持**：为 Godot、Unity、Unreal Engine 5 三大主流游戏引擎提供了专门的代理和工具集，确保技术建议的针对性和准确性。

适用场景：主要面向独立游戏开发者、小型团队或对游戏开发感兴趣但缺乏完整团队支撑的创作者。当他们在使用 Claude Code 进行从零开始的游戏原型设计、系统搭建或复杂功能开发时，可以利用此框架获得类似专业工作室的结构化协作与审查支持。

技术亮点：其核心并非复杂的代码库（项目主要使用 Shell 脚本），而是一套精心设计的“提示工程”与“AI 代理编排”系统。它通过 `.claude` 目录下的配置文件（代理、技能、规则、钩子），深度定制和扩展了 Claude Code 的会话行为，将单一模型能力“拆分”并组织成协同工作的专家团队，这是一种新颖的 AI 应用架构模式。

上手建议：新用户应从阅读完整的 README 开始，重点关注“Studio Hierarchy”和“Slash Commands”部分，理解代理分工。然后，在 Claude Code 中导入该项目配置，尝试使用 `/start` 或 `/help` 命令来引导初始会话。对于贡献者，可以研究 `.claude/` 目录下的各种配置模板，通过修改或新增代理、技能、规则来扩展框架能力。

#7 TypeScript 连续 4 天

voicebox

The open-source voice synthesis studio

Stars **19,477** Forks **2,244** Today **+880**

https://github.com/jamiepine/voicebox

项目简介：Voicebox 是一个开源的、本地优先的语音合成与克隆工作室。它旨在为用户提供一个功能强大且完全私密的替代方案，以解决对 ElevenLabs 等商业云服务的依赖，让用户能在自己的电脑上完成高质量的语音克隆、生成和处理。

核心功能：1. **多引擎语音克隆与合成**：集成 Qwen3-TTS、LuxTTS 等 5 个 TTS 引擎，支持 23 种语言。2. **本地化与隐私**：所有模型和音频数据均在本地运行，确保数据安全。3. **专业音频处理**：提供时间线编辑器、多音轨故事编排以及音高、混响等后期效果。4. **API 优先设计**：提供 REST API，便于开发者将语音功能集成到自己的应用中。

适用场景：**内容创作者**（如制作多角色有声书、播客旁白）、**独立开发者**（为应用添加本地语音交互功能）以及**注重隐私的个人用户**（需要生成个性化语音但不愿上传数据到云端）都会用到它。

技术亮点：1. **架构选择**：采用 Tauri（Rust）框架而非 Electron 构建桌面应用，显著提升了性能和资源效率。2. **模型集成**：巧妙地整合了多个前沿的开源 TTS 模型（如 Qwen、Chatterbox），提供了多样化的语音生成选择。3. **跨平台支持**：全面支持 macOS（MLX/Metal）、Windows（CUDA）、Linux 及多种硬件加速方案，展现了优秀的工程适配能力。

上手建议：普通用户可直接从官网 voicebox.sh 下载对应平台的安装包体验。开发者若想集成其语音能力，应从查阅官方文档的 **API 部分** 开始；有意贡献代码则建议先阅读项目源码（TypeScript）并关注 Issues 列表。

#8 TypeScript NEW

craft-agents-oss

Stars **4,174** Forks **622** Today **+107**

https://github.com/lukilabs/craft-agents-oss

项目简介：Craft Agents 是一个开源的“Agent-Native”（智能体原生）桌面应用，旨在为开发者提供一个直观、强大的界面来与以 Claude 为代表的大语言模型智能体协同工作。它解决了传统 CLI 或代码编辑器在管理多个 AI 代理、连接外部服务（API、MCP）以及组织复杂工作流时不够直观和高效的问题。

核心功能：1. **零配置连接**：通过自然语言指令即可连接 Linear、Slack 等服务的 API 或 MCP 服务器，自动处理凭证和配置。2. **多会话与工作流管理**：提供类似收件箱的多会话界面，支持自定义状态流（如 Todo/Done），便于任务管理。3. **多模型与多源支持**：可同时接入 Anthropic、OpenAI、Google AI Studio 等多个 LLM 提供商，并混合使用 MCP 服务器、REST API 等多种数据源。4. **技能（Skills）系统**：支持通过描述快速创建、导入和管理可复用的智能体指令和工具。

适用场景：主要面向**需要频繁与 AI 智能体交互的开发者、产品经理或技术团队**，尤其适用于需要将 AI 能力与真实数据源（如公司内部的 Jira、数据库、邮件）结合，进行自动化任务处理、数据分析或复杂工作流编排的场景。

技术亮点：1. **基于 TypeScript 和 Electron**，实现了跨平台的桌面体验。2. 深度整合 **Claude Agent SDK 和 Pi SDK**，并在此基础上构建了更优的 UX。3. 践行“Agent-Native”理念，其自身也完全使用 Craft Agents 来开发和定制，实现了“用智能体构建智能体工具”的闭环。4. 支持 **Stdio 本地 MCP 服务器**，能无缝运行本地脚本或工具。

上手建议：对于使用者，强烈推荐通过官网提供的**一行安装脚本**快速体验，并观看演示视频了解其核心交互模式。对于贡献者，项目使用 Bun 作为运行时，克隆仓库后运行 `bun install` 和 `bun run electron:start` 即可开始开发，可以从修复 Good First Issue 或熟悉其模块化架构入手。

#9 Python NEW

opensre

Build your own AI SRE agents. The open source toolkit for the AI era ✨

Stars **1,102** Forks **147** Today **+167**

https://github.com/Tracer-Cloud/opensre

项目简介：OpenSRE 是一个开源框架，旨在帮助用户构建和训练自己的 AI SRE（站点可靠性工程）智能体。它要解决的核心问题是：当生产环境发生故障时，证据（日志、指标、链路追踪等）分散各处，人工排查效率低下且复杂。该项目试图为AI驱动的生产环境故障诊断和响应，提供一个可训练、可评估的“强化学习环境”。

核心功能：1. **可定制AI SRE智能体**：提供易于部署的智能体，用于自动化事故调查和响应。2. **合成与真实测试套件**：包含模拟的故障场景（synthetic）和基于真实云环境（如K8s、AWS）的端到端测试，用于训练和评估智能体。3. **多工具集成**：能够连接超过40种现有运维工具（如Grafana、Datadog、Slack），并定义自定义工作流。4. **结构化调查流程**：从告警触发到根因分析、报告生成、建议修复，实现自动化闭环。

适用场景：主要面向**SRE工程师、运维团队和平台工程师**。适用于需要快速、自动化地调查和响应**云原生基础设施、分布式系统生产环境故障**的场景，特别是那些告警频繁、排障链路长且复杂的组织。

技术亮点：1. **将“强化学习环境”理念引入运维领域**：项目不只是一个工具集，更致力于构建一个带有评估标准的“训练场”，这是其区别于普通自动化脚本的关键。2. **强调真实性与可评估性**：通过合成故障模拟和真实的端到端云测试，为AI智能体提供高质量、贴近实战的训练数据与反馈机制。3. **架构开放性**：支持连接广泛的现有工具链，并允许用户自定义工作流，避免了厂商锁定。

上手建议：对于**使用者**，建议从官方文档的 Quickstart 开始，使用 `opensre onboard` 命令进行初始配置，并尝试用项目自带的测试告警文件（如K8s示例）运行一次调查流程。对于**贡献者**，可以详细阅读 SETUP.md 搭建开发环境，并从理解测试套件（`tests/` 目录）的结构和编写新的故障模拟场景入手。

#10 Shell NEW

superpowers

An agentic skills framework & software development methodology that works.

Stars **157,028** Forks **13,646** Today **+2,058**

https://github.com/obra/superpowers

项目简介：Superpowers 是一个面向 AI 编程助手（如 Claude、Cursor）的“技能框架”与软件开发方法论。它旨在解决当前 AI 助手直接生成代码时可能出现的需求不明确、设计缺失、代码质量不可控等问题，通过一套结构化的流程来引导和规范 AI 的开发行为。

核心功能：1. **需求澄清与设计**：在编码前，通过对话引导用户明确需求，并产出可审阅的设计文档。2. **子代理驱动开发**：将任务分解后，由独立的“子代理”执行并交叉审查，实现自动化、可监督的编码流程。3. **强制测试驱动开发**：内置严格的 RED-GREEN-REFACTOR 循环，确保代码质量。4. **可组合的技能库**：提供如系统化调试、Git 工作树管理等模块化技能，供 AI 在适当时机自动调用。

适用场景：主要面向依赖 AI 编程助手（如 Claude Code、Cursor、GitHub Copilot）进行软件开发的工程师或团队。适用于需要 AI 协助完成从需求分析到代码实现、测试的完整功能开发，尤其适合希望提升 AI 产出代码的可预测性、可维护性和工程规范性的场景。

技术亮点：其核心并非复杂的算法，而是一种精巧的“元编程”设计。它将软件开发的最佳实践（如 TDD、YAGNI）和工程流程（如代码审查、分支管理）封装成一系列可被 AI 识别和自动触发的“技能”指令，本质上是在 LLM 之上构建了一个确定性的、强约束的代理行为框架。

上手建议：对于使用者，最直接的方式是参照 README，根据自己常用的 AI 编码工具（如 Claude Code 或 Cursor）通过其插件市场安装 Superpowers 插件即可体验。对于贡献者或深度用户，可以仔细研究其技能库（Skills Library）的构成与触发逻辑，理解其如何通过 Shell 脚本和提示词工程来塑造 AI 的行为模式。

数据来源：github.com/trending

报告日期：2026-04-17

由 DeepSeek AI 提供智能分析 | 自动生成